

Base de Données

AI Test Automation Framework

Documentation detaillée pour la soutenance

Rapporteur et President - Experts en bases de données

Projet NoveoCare

Mariam El Houche-Ext

Table des matieres

1. Presentation de la base de données

- 1.1 Technologie et choix
- 1.2 Architecture de la couche données
- 1.3 Principe dual : SQLAlchemy ORM + Core

2. Schema des 5 tables

- 2.1 Table healing_events
- 2.2 Table metrics_snapshots
- 2.3 Table cucumber_runs
- 2.4 Table test_runs
- 2.5 Table app_users

3. Index et performance

- 3.1 Index definis
- 3.2 Strategies d'indexation
- 3.3 Analyse des requêtes critiques

4. Repositories et accès aux données

- 4.1 Pattern Repository
- 4.2 CucumberRunRepository
- 4.3 HealingEventRepository
- 4.4 MetricsSnapshotRepository
- 4.5 TestRunRepository (synchrone)
- 4.6 UserRepository

5. Mecanismes de migration et schema

- 5.1 Initialisation via schema.sql
- 5.2 Création automatique via SQLAlchemy
- 5.3 Migrations dynamiques (ensure_schema_compatibility)
- 5.4 Fallback SQLite pour les tests

6. Flux de données details

- 6.1 Insertion des resultats Cucumber
- 6.2 Insertion des événements de healing
- 6.3 Calcul et stockage des métriques
- 6.4 Resolution des métriques (fallback 3 niveaux)
- 6.5 Gestion des test runs

7. Q&A - Questions possibles

- 7.1 Questions sur le choix de la technologie
- 7.2 Questions sur le schema et la modelisation
- 7.3 Questions sur les performances
- 7.4 Questions sur les migrations et l'evolution
- 7.5 Questions sur les tests et la fiabilite
- 7.6 Questions avancees (expert)

1. Presentation de la base de données

1.1 Technologie et choix

La base de données du projet AI Test Automation Framework utilise PostgreSQL 16 en production et SQLite comme fallback pour les tests unitaires. Ce double support est géré de manière transparente par SQLAlchemy 2.0, l'ORM (Object-Relational Mapping) choisi pour le projet.

Raisons du choix de PostgreSQL :

- Robustesse et maturité : PostgreSQL est un SGBD relationnel open-source éprouvé, avec un excellent support des types de données avancés (TIMESTAMPTZ, JSON, Numeric precis).
- Performance analytique : les fonctions d'agrégation (COUNT, AVG, SUM) utilisées pour les métriques de healing sont optimisées par l'optimiseur de requêtes PostgreSQL.
- Containerisable : l'image officielle postgres:16-alpine est légère (~200MB) et facile à déployer dans la stack Docker.
- Extensions : possibilité d'ajouter des indexes avancés (GIN, BRIN) ou des fonctions de fenêtrage pour les tendances historiques.

Raisons du choix de SQLAlchemy 2.0 :

- Support dual synchrone/asynchrone : le projet utilise le package databases (asynchrone) pour les opérations CRUD principales et SQLAlchemy Core (synchrone) pour le repository TestRunRepository qui utilise un engine synchrone.
- Mappings déclaratifs : les modèles SQLAlchemy (DeclarativeBase) permettent de définir les tables en Python avec une syntaxe claire et typée.
- Compatibilité PostgreSQL/SQLite : SQLAlchemy abstrait les différences de dialecte. Par exemple, la migration dynamique (ensure_schema_compatibility) détecte 'sqlite' dans l'URL et ignore les ALTER TABLE non supportés.

Explication synchrone vs asynchrone

Un accès synchrone à la base de données signifie que le thread d'exécution est bloqué pendant l'attente de la réponse de la base. Si l'opération prend 200ms, le thread ne peut rien faire d'autre pendant ce temps. Cela pose problème dans un serveur web comme FastAPI qui gère des centaines de connexions simultanées avec un nombre limité de threads (workers).

Un accès asynchrone (async/await en Python) permet de libérer le thread pendant l'attente de la réponse. Le thread peut traiter d'autres requêtes pendant que la base traite la précédente. Cela améliore considérablement le débit (throughput) du serveur sans multiplier le nombre de threads.

Comparaison avec d'autres approches :

Approche	Modèle	Performance	Complexité
psycopg2 (synchrone pur)	Synchrone bloquant	Fiable, bloque le thread	Faible
SQLAlchemy Core + databases	Async avec await	Non-bloquant, haut débit	Moyenne
SQLAlchemy ORM + AsyncSession	Async avec session	Performant, overhead ORM	Élevée
Django ORM	Synchrone (Celery pour async)	Bloquant, nécessite file	Moyenne
Tortoise ORM	Asynchrone natif	Bon, moins mature	Moyenne
AIOHTTP + asyncpg	Asynchrone sans ORM	Très performant	Élevée

Notre projet fait un choix hybride original : les 4 repositories principaux utilisent databases (async) pour ne pas bloquer FastAPI, tandis que TestRunRepository utilise un engine synchrone car il est manipulé depuis des threads standards (threading.Thread pour les exécutions Maven) qui ne peuvent pas utiliser async/await. Ce compromis offre le meilleur des deux mondes sans la complexité d'un ORM complet.

1.2 Architecture de la couche données

La couche d'accès aux données suit une architecture en 3 couches :

Couche 1 - Schemas (Pydantic) : validation des entrées/sorties API

```
src/schemas/dashboard_schemas.py  
src/schemas/user_schemas.py
```

Couche 2 - Modèles (SQLAlchemy ORM) : mapping objet-relationnel

```
src/models/base.py (DeclarativeBase)  
src/models/dashboard_model.py (4 tables)  
src/models/user_model.py (1 table)
```

Couche 3 - Repositories : accès aux données (CRUD + requêtes métier)

```
src/repositories/cucumber_repository.py  
src/repositories/healing_repository.py  
src/repositories/metrics_repository.py  
src/repositories/test_run_repository.py  
src/repositories/user_repository.py
```

Chaque couche a une responsabilité unique : les schemas valident le format des données, les modèles définissent la structure des tables, les repositories encapsulent les requêtes SQL. Les services (src/services/) orchestrent la logique métier en appelant les repositories.

1.3 Dualité asynchrone/synchrone

Le projet utilise deux modes d'accès à la base de données :

- Async (databases) : HealingEventRepository, MetricsSnapshotRepository, CucumberRunRepository, UserRepository utilisent databases.Database asynchrone avec des requêtes SQLAlchemy Core. Cela permet de ne pas bloquer le thread principal de FastAPI pendant les opérations I/O.
- Sync (SQLAlchemy Engine) : TestRunRepository utilise sqlalchemy.create_engine synchrone avec des connexions directes (with engine.begin() as conn). Ce choix est lié à l'utilisation de la bibliothèque standard threading pour gérer les exécutions Maven en arrière-plan, et l'écriture des logs JSON qui nécessite des opérations de sérialisation synchrones.

Les deux modes coexistent via le fichier src/core/database.py qui initialise à la fois l'instance databases.Database et l'engine SQLAlchemy. Les tables sont définies une seule fois dans les modèles et partagées entre les deux modes.

2. Schema des 5 tables

2.1 Table healing_events

Stocke chaque tentative de healing (auto-reparation) d'un localisateur Selenium. C'est la table la plus riche du projet avec 18 colonnes.

Colonne	Type SQL	Modèle Python	Description
id	SERIAL	Integer, PK	Identifiant unique
created_at	TIMESTAMPZ	DateTime(timezone=True)	Horodatage de l'événement
scenario_name	VARCHAR	String	Nom du scénario Cucumber concerne
old_locator_type	VARCHAR(100)	String	Type de l'ancien locator (xpath, css, id...)
old_locator_val	TEXT	Text	Valeur de l'ancien locator
success	BOOLEAN	Boolean, NOT NULL	Succes ou echec de la guerison
score	NUMERIC(10,4)	Numeric	Score final de confiance (0-1)
structural_score	NUMERIC(10,4)	Numeric	Score de similarite structurelle
semantic_score	NUMERIC(10,4)	Numeric	Score de similarite sémantique
new_locator_type	VARCHAR(100)	String	Type du nouveau locator propose
new_locator_val	TEXT	Text	Valeur du nouveau locator
healing_time_ms	INTEGER	Integer	Temps de resolution en ms
baseline_hit	BOOLEAN	Boolean, default=False	Resolu via le cache baseline ?
elements_extracted	INTEGER	Integer	Nb d'éléments extraits du DOM
after_struct_filter	INTEGER	Integer	Nb d'éléments après filtre structurel
after_spatial_filter	INTEGER	Integer	Nb d'éléments après filtre spatial
sent_to_nlp	INTEGER	Integer	Nb d'éléments envoyes au NLP
error_message	TEXT	Text	Message d'erreur si echec
exception_type	VARCHAR(64)	String(64)	Type d'exception Java (NoSuchElementException...)
run_id	VARCHAR(128)	String(128)	Identifiant du run de test associe

Les colonnes elements_extracted, after_struct_filter, after_spatial_filter, sent_to_nlp retracent le pipeline de healing : extraction DOM -> filtre structurel -> filtre spatial -> NLP. Ces compteurs permettent de diagnostiquer l'efficacite de chaque étape du pipeline.

2.2 Table metrics_snapshots

Stocke des instantanes periodiques des métriques globales de healing. Contrairement a healing_events qui stocke des événements individuels, metrics_snapshots stocke des valeurs agrégées (taux, moyennes) calculees par le notebook Colab ou derivees des événements.

Colonne	Type SQL	Description
id	SERIAL PK	Identifiant unique
captured_at	TIMESTAMPZ	Horodatage de la capture
total_healing_requests	INTEGER	Nombre total de requêtes de healing
successful_healings	INTEGER	Guerisons reussies
failed_healings	INTEGER	Guerisons echouees
baseline_hits	INTEGER	Guerisons via baseline cache
total_elements_extracted	INTEGER	Total éléments extraits du DOM
total_after_struct	INTEGER	Après filtre structurel
total_after_spatial	INTEGER	Après filtre spatial
total_sent_to_nlp	INTEGER	Envoyes au NLP

total_healing_time_ms	BIGINT	Temps total de healing en ms
healing_rate	NUMERIC(6,4)	Taux de guerison (successful/total)
baseline_hit_rate	NUMERIC(6,4)	Taux d'utilisation du baseline
avg_healing_time_ms	NUMERIC(12,2)	Temps moyen de guerison
avg_final_score	NUMERIC(8,4)	Score moyen pondere
avg_structural_score	NUMERIC(8,4)	Score structurel moyen
avg_semantic_score	NUMERIC(8,4)	Score semantique moyen
nlp_filter_efficiency	NUMERIC(6,4)	Ratio éléments evites / éléments extraits
run_id	VARCHAR(128)	Run de test associe

NUMERIC(6,4) permet des taux sur 4 decimales (ex: 0.9534). BIGINT pour total_healing_time_ms car la somme peut etre grande.

Note : les colonnes run_id sont partagees entre les 3 tables principales pour permettre les jointures et le filtrage par run.

2.3 Table cucumber_runs

Stocke les resultats de chaque scénario Cucumber exécuté. Chaque ligne represente un scénario unique (feature + nom) avec son statut et sa duree.

Colonne	Type SQL	Description
id	SERIAL PK	Identifiant unique
run_at	TIMESTAMP TZ	Horodatage de l'exécution
feature_name	VARCHAR(255)	Nom de la feature Cucumber
scénario	VARCHAR(255)	Nom du scénario
status	VARCHAR(32)	passed, failed, skipped, flaky
duration_ns	BIGINT	Duree en nanosecondes
tags	TEXT	Tags Cucumber (ex: @login, @smoke)
run_id	VARCHAR(128)	Run de test associe
classification	VARCHAR(32)	Classification après rerun : passed/flaky/failed

duration_ns est stocke en nanosecondes (precision Java System.nanoTime()) et converti en millisecondes au niveau API (division par 1 000 000). La colonne classification est renseignee par le TestClassifier après re-exécution des scénarios echoues.

2.4 Table test_runs

Stocke les executions de test lancees via le dashboard (TestRunManager). Contrairement aux autres tables, elle utilise run_id comme cle primaire directe (pas de SERIAL auto-increment).

Colonne	Type SQL	Description
run_id	VARCHAR(128) PK	UUID hex (12 caracteres) généré par le manager
tags	VARCHAR	Tags Cucumber passes a l'exécution
runner	VARCHAR	Classe runner utilisée
suite_name	VARCHAR	Nom de la suite de tests
status	VARCHAR(32)	pending, running, completed, failed, cancelled
created_at	TIMESTAMP TZ	Date de création
completed_at	TIMESTAMP TZ NULLABLE	Date de fin (NULL si en cours)
logs	TEXT	Logs d'exécution (JSON array serialise)
exit_code	INTEGER NULLABLE	Code de sortie Maven
error	TEXT NULLABLE	Message d'erreur si echec

La colonne logs stocke un tableau JSON serialize (ex: ["[runner] Starting Maven...", "[runner] Tests completed"])). Le repository TestRunRepository utilise json.dumps/json.loads pour la serialisation/deserialisation. Ce choix evite une table séparée de logs et simplifie le stockage.

2.5 Table app_users

Stocke les utilisateurs du dashboard avec leur role et leur mot de passe hache.

Colonne	Type SQL	Description
id	SERIAL PK	Identifiant unique
email	VARCHAR(255) UNIQUE NOT NULL	Email de l'utilisateur
password_hash	VARCHAR(255) NOT NULL	Hash bcrypt du mot de passe
display_name	VARCHAR(255) NOT NULL	Nom affiche
role	VARCHAR(32) NOT NULL	admin, project_manager, qa_engineer
is_active	BOOLEAN DEFAULT TRUE	Compte actif ou desactive
created_at	TIMESTAMP TZ	Date de création
updated_at	TIMESTAMP TZ	Date de dernière modification

Le mot de passe est hache avec bcrypt via la bibliotheque passlib. Chaque hash est unique (sel sale) meme pour deux mots de passe identiques. La contrainte CHECK (role IN (...)) est definie dans le schema.sql mais pas dans le modèle SQLAlchemy (la validation est faite par Pydantic dans le schema UserCreate/UserUpdate).

3. Index et performance

3.1 Index definis

Les indexes sont definis dans le fichier db/schema.sql et complètes automatiquement par la méthode `ensure_schema_compatibility()`. Voici la liste complète :

Table	Index	Colonne(s)	Type
healing_events	idx_healing_events_created_at	created_at DESC	BTREE
healing_events	idx_healing_events_run_id	run_id	BTREE
metrics_snapshots	idx_metrics_snapshots_captured_at	captured_at DESC	BTREE
metrics_snapshots	idx_metrics_snapshots_run_id	run_id	BTREE
cucumber_runs	idx_cucumber_runs_run_at	run_at DESC	BTREE
cucumber_runs	idx_cucumber_runs_status	status	BTREE
cucumber_runs	idx_cucumber_runs_run_id	run_id	BTREE
app_users	app_users_email_key	email (UNIQUE)	UNIQUE BTREE

3.2 Explication des types d'index (BTREE)

BTREE (Balanced Tree) est la structure d'index par défaut dans PostgreSQL et dans la plupart des SGBD relationnels. Il s'agit d'un arbre équilibre qui stocke les valeurs dans un ordre trié (ASC ou DESC) et permet les opérations suivantes en temps $O(\log n)$ (logarithmique) :

- Recherche par égalité : `WHERE run_id = 'abc123'` -> trouve l'enregistrement en quelques étapes, même avec des millions de lignes.
- Recherche par plage : `WHERE created_at > '2025-01-01'` -> parcourt séquentiellement les feuilles de l'arbre.
- Tri : `ORDER BY created_at DESC` -> lit les feuilles dans l'ordre, pas de sort supplémentaire.
- Agrégation avec `GROUP BY` : utilisé le parcours ordonné pour les fonctions `COUNT`, `SUM`, `AVG`.

Sans index, PostgreSQL doit faire un Sequential Scan (parcours intégral) de toute la table, ce qui est $O(n)$ et devient très lent avec le volume. Avec un index BTREE, la recherche passe en $O(\log n)$. Par exemple, pour trouver un enregistrement parmi 1 million de lignes, un index BTREE nécessite environ 20 étapes ($\log_2(1\,000\,000) \approx 20$) au lieu de 1 million.

Les indexes DESC sont déclarés explicitement dans le `schema.sql` car la plupart des requêtes trient par date décroissante (du plus récent au plus ancien). Un index DESC évite à PostgreSQL de parcourir l'index à l'envers, ce qui offre un gain de performance marginal mais systématique.

Analyse des indexes du projet

Les indexes DESC sur `created_at/captured_at/run_at` optimisent la requête `'ORDER BY ... DESC LIMIT 1'` qui récupère le dernier enregistrement (utilisée par `get_latest_metrics()`, `get_latest_run_id()`, etc.). L'index sur `run_id` est utilisé par toutes les requêtes filtrées par `run` (`get_counts_by_run_id()`, `fetch_by_run_id()`, etc.). L'index sur `status` est utilisé par les requêtes d'agrégation `GROUP BY status` (`get_counts_by_run_id()` dans `cucumber_summary()`).

3.3 Requêtes critiques et leur plan

Requête la plus fréquente : récupération du dashboard

```
-- Résumé des résultats Cucumber pour un run
SELECT status, COUNT(*) as cnt
FROM cucumber_runs
WHERE run_id = 'abc123' AND run_id IS NOT NULL
GROUP BY status;
-> Utilisé l'index idx_cucumber_runs_run_id, pas de seq scan
```

Requête la plus coûteuse : historique des métriques


```
-- Historique des snapshots (dashboard: historique des tendances)
SELECT * FROM metrics_snapshots
WHERE run_id = 'abc123'
ORDER BY captured_at ASC
LIMIT 50;
-> Utilisé l'index idx_metrics_snapshots_run_id
```

Requête de calcul de métriques :

```
-- Calcul du healing_rate et avg_final_score (fallback healing_events)
SELECT COUNT(*), SUM(success::int), AVG(score), ...
FROM healing_events
WHERE run_id = 'abc123';
-> Utilisé l'index idx_healing_events_run_id
```

4. Repositories et accès aux données

4.1 Pattern Repository

Le pattern Repository est utilisé pour encapsuler l'accès aux données. Chaque table a son propre repository qui expose des méthodes métier (`create_scenario()`, `get_latest_metrics()`, `fetch_by_run_id()`, etc.). Les repositories ne connaissent pas la couche service : ils reçoivent des paramètres simples et retournent des dictionnaires Python.

4.2 CucumberRunRepository

Méthodes exposées :

- `create_scenario(run_at, feature_name, scénario, status, duration_ns, tags, run_id, classification)` : insère un résultat de scénario.
- `get_latest_run_id()` : `SELECT MAX(run_id)` avec tri par `run_at` DESC, retourné le dernier `run_id` non-null.
- `get_latest_run_at(run_id)` : `SELECT MAX(run_at)` pour un `run_id` donné.
- `get_global_latest_run_at()` : `SELECT MAX(run_at)` global, utilisé quand aucun `run_id` n'est fourni.
- `get_counts_by_run_id(run_id)` : `SELECT status, COUNT(*) GROUP BY status` pour un run. Utilisé par `cucumber_summary()` pour calculer `passed/failed/skipped/total`.
- `get_avg_duration(run_id)` : `SELECT AVG(duration_ns)` pour le temps moyen par scénario.
- `get_recent(run_id, limit)` : `SELECT * FROM cucumber_runs ORDER BY run_at DESC LIMIT limit`. Utilisé par la page Tests du dashboard.

4.3 HealingEventRepository

Méthodes exposées :

- `create(event_data)` : insère un événement avec `created_at = now(UTC)`.
- `list(limit, offset, run_id)` : pagination avec tri par `created_at` DESC, filtre optionnel par `run_id`.
- `fetch_all()` : toutes les données, utilisé par l'agrégation de métriques dérivées.
- `fetch_by_run_id(run_id)` : tous les événements d'un run spécifique.
- `fetch_with_null_run_id()` : événements sans `run_id` (cas legacy).
- `fetch_recent(run_id, cutoff, limit)` : événements récents avec cut-off temporel (`run_at - 3h`) pour éviter les doublons entre runs.

4.4 MetricsSnapshotRepository

Méthodes exposées :

- `create(snapshot_data)` : insère un snapshot avec `captured_at = now(UTC)`.
- `get_latest(run_id)` : récupère le snapshot le plus récent (`ORDER BY captured_at DESC LIMIT 1`), filtre optionnel par `run_id`.
- `get_history(run_id, limit)` : historique trié par `captured_at` ASC pour le graphique d'évolution. Limite par défaut à 50 points.
- `fetch_all()`, `fetch_by_run_id(run_id)`, `fetch_with_null_run_id()` : mêmes patterns que `HealingEventRepository`.

4.5 TestRunRepository (synchrone)

Ce repository utilise un engine SQLAlchemy synchrone (`sqlalchemy.create_engine`) contrairement aux autres qui utilisent `databases.Database` asynchrone.

- `upsert(run_data)` : `INSERT OR UPDATE (upsert)`. Vérifié l'existence par `run_id`, exécuté `UPDATE` si existant, `INSERT` sinon. Les logs sont serialisés en JSON.
- `get(run_id)` : `SELECT` avec désérialisation JSON des logs et normalisation des datetime.
- `list(limit, offset)` : `SELECT` paginé avec `ORDER BY created_at DESC`. Utilisé par `TestRunManager.list_runs()` qui merge les données mémoire et BDD.

4.6 UserRepository

Repository asynchrone pour la gestion des utilisateurs. Utilisé le schema Pydantic User pour le typage des retours (find_by_email, find_by_id retournent des instances User).

- find_by_email(email) / find_by_id(user_id) : recherche par email ou id, retourné User ou None.
- create(email, password_hash, display_name, role) : création avec horodatage.
- update(user_id, updates) : mise a jour partielle, leve HTTPException 404 si inexistant.
- delete(user_id) : suppression physique.
- change_password(user_id, password_hash) : mise a jour du hash uniquement.
- list_all() : tous les utilisateurs tries par created_at ASC.

5. Mecanismes de migration et schema

5.1 Initialisation via schema.sql

Le fichier db/schema.sql est monte dans le conteneur PostgreSQL via docker-compose.yml comme script d'initialisation :

```
services:
  postgres:
    ...
  volumes:
    - ./db/schema.sql:/docker-entrypoint-initdb.d/schema.sql
```

Au démarrage du conteneur, PostgreSQL exécuté automatiquement tous les scripts dans /docker-entrypoint-initdb.d/ par ordre alphabétique, avant d'accepter les connexions. Ce mécanisme garantit que les tables existent avant le démarrage de l'API.

5.2 Création automatique via SQLAlchemy

Au démarrage de l'API FastAPI, une deuxième couche de création est exécutée :

```
@app.on_event("startup")
async def startup():
    metadata.create_all(engine)          # Tables healing_events, metrics_snapshots,
                                         #   cucumber_runs, test_runs
    UserModel.metadata.create_all(engine) # Table app_users
    _run_schema_compat()                 # Migrations ALTER TABLE
    await database.connect()
    await _seed_default_users()          # 3 comptes par défaut
```

metadata.create_all(engine) utilise SQLAlchemy pour créer les tables si elles n'existent pas encore. Cette approche est idempotente : si la table existe déjà, elle n'est pas recréée. Cela permet au schema.sql et à SQLAlchemy de coexister.

5.3 Migrations dynamiques

La méthode DashboardService.ensure_schema_compatibility() gère les évolutions du schéma au fil des versions. Au lieu d'utiliser un outil de migration (Alembic), le projet exécute des ALTER TABLE ADD COLUMN IF NOT EXISTS au démarrage :

```
@staticmethod
def ensure_schema_compatibility(engine):
    statements = [
        "ALTER TABLE healing_events ADD COLUMN IF NOT EXISTS exception_type VARCHAR(64)",
        "ALTER TABLE healing_events ADD COLUMN IF NOT EXISTS run_id VARCHAR(128)",
        "ALTER TABLE metrics_snapshots ADD COLUMN IF NOT EXISTS run_id VARCHAR(128)",
        "ALTER TABLE cucumber_runs ADD COLUMN IF NOT EXISTS run_id VARCHAR(128)",
        "CREATE INDEX IF NOT EXISTS idx_healing_events_run_id ON healing_events (run_id)",
        "CREATE INDEX IF NOT EXISTS idx_metrics_snapshots_run_id ON metrics_snapshots (run_id)",
        "CREATE INDEX IF NOT EXISTS idx_cucumber_runs_run_id ON cucumber_runs (run_id)",
        "ALTER TABLE cucumber_runs ADD COLUMN IF NOT EXISTS classification VARCHAR(32)",
    ]
    with engine.begin() as conn:
        for sql in statements:
            conn.execute(sqlalchemy.text(sql))
```

Cette approche est délibérément simple et adaptée à un projet de cette taille. Pour un projet en production avec plusieurs environnements, Alembic serait recommandé car il permet des rollbacks structurés et un versionnage explicite des migrations.

5.4 Fallback SQLite pour les tests

Quand l'URL de base de données contient "sqlite" (détecté par la comparaison case-insensitive de l'URL), plusieurs comportements changent :

- Tables creees immediatement : database.py executé create_all pour toutes les tables au moment de l'import (pas besoin du lifecycle FastAPI).
- Migrations ignorees : _run_schema_compat() ne fait rien ('if sqlite in str(engine.url).lower(): return').
- Tests isolees : le module test_auth.db est un fichier SQLite dedie aux tests d'authentification.
- Seed users integre : les comptes par défaut sont crees via _seed_default_users() meme en SQLite.

6. Flux de données détaillés

6.1 Insertion des resultats Cucumber

Les resultats Cucumber sont inseres via deux chemins :

Chemin 1 : Dashboard Test Runner (Python)

Le `TestRunManager.execute_maven_run()` exécuté Maven avec `DashboardCucumberRunner`, puis appelle `dashboard_api.push_cucumber_json()` qui parse le fichier `cucumber-{run_id}.json` et appelle `POST /api/cucumber-runs` pour chaque scénario.

```
Thread Maven -> cucumber-{run_id}.json
-> parse JSON (feature.éléments)
-> compute_scenario_status (steps + hooks)
-> POST /api/cucumber-runs -> create_scenario() x N
```

Chemin 2 : CI/CD DashboardPushMain (Java)

`DashboardPushMain` est exécuté par le plugin `exec-maven-plugin` pendant la phase test. Il lit `target/cucumber.json` et appelle l'API REST du dashboard.

```
Maven test -> cucumber.json
-> DashboardPushMain.java
-> POST /api/cucumber-runs (machine-to-machine via X-API-Key)
```

6.2 Insertion des événements de healing

Chaque tentative de healing est insérée via `DashboardReporter.pushHealingEvent()` qui appelle `POST /api/healing-events`. Les champs envoyés incluent tous les scores, les compteurs du pipeline (`elements_extracted`, `after_struct_filter`, ...) et le type d'exception Java.

```
BasePage.findElement() -> NoSuchElementException
-> callHealingAPI() -> POST /heal
-> pushHealingEvent()
-> POST /api/healing-events
-> HealingEventRepository.create() -> INSERT INTO healing_events
```

6.3 Calcul et stockage des métriques

Les snapshots de métriques sont insérés via `POST /api/metrics` par le notebook Colab. Chaque snapshot contient les valeurs agrégées calculées sur l'ensemble des données.

Si aucun snapshot n'est disponible (fallback), `DashboardService.aggregate_metrics()` calcule les métriques à partir des données brutes de `healing_events` :

```
healing_rate = successful_healings / total_healing_requests
baseline_hit_rate = baseline_hits / total
avg_healing_time_ms = total_healing_time_ms / total
avg_final_score = AVG(score) sur les événements avec score non-null
nlp_filter_efficiency = 1 - (total_sent_to_nlp / total_elements_extracted)
```

6.4 Resolution des métriques (fallback 3 niveaux)

La méthode `resolve_healing_payload()` implemente un fallback en 3 étapes :

Niveau 1 : `metrics_snapshots` (table)

- Cherche le snapshot le plus récent pour le `run_id` donné.

- Requête : `SELECT * FROM metrics_snapshots WHERE run_id = ? ORDER BY captured_at DESC LIMIT 1.`
- Si trouve, retourné les métriques + historique + événements récents.

Niveau 2 : agrégation depuis `healing_events` (table)

- Si aucun snapshot, agrégé les événements de healing bruts.
- Requête : `SELECT COUNT(*), SUM(success), AVG(score), ... FROM healing_events WHERE run_id = ?.`
- Calcule `healing_rate`, `baseline_hit_rate`, `avg_healing_time_ms`, etc. via `aggregate_metrics()`.

Niveau 3 : endpoint externe (`SELF_HEALING_METRICS_URL`)

- Si aucune donnée en base, appelle l'API externe (notebook Colab /metrics).
- Requête HTTP GET avec timeout configurable (5s par défaut).
- Normalise la réponse via `normalize_external_metrics()`.

Le scope de recherche suit aussi un fallback : `run_id` spécifique -> `run_id=NULL` -> n'importe quel run (latest_available).

6.5 Gestion des test runs

TestRunManager gère les exécutions de test de manière duale :

- Mémoire vive : dictionnaire `_test_runs` (dict[str, dict]) avec verrou `threading.Lock()` pour la concurrence.
- Persistance BDD : `TestRunRepository.upsert()` sauvegarde le run à la fin de l'exécution (status, logs, exit_code).
- Merge à la lecture : `list_runs()` fusionne les runs mémoire et BDD, déduplication par `run_id`, tri par `created_at` DESC, pagination.

```
1. create_run() -> généré UUID, stocke dans _test_runs[run_id]
2. execute_maven_run() -> status=running, lance Maven
3. A chaque ligne de log -> _append_log(run_id, line)
4. Fin exécution -> status=completed/failed, persist en BDD
5. list_runs() -> merge mémoire + BDD, tri DESC, pagination
```

7. Q&A - Questions possibles

Cette section contient les questions probables lors de la soutenance, classees par theme et par niveau de difficulte. Les questions marquées avec un asterisque sont des questions expertes pour les rapporteurs specialises en bases de données.

7.1 Questions sur le choix de la technologie

Q: Pourquoi PostgreSQL plutot que MySQL ou MariaDB ?

R: PostgreSQL offre un meilleur support des types temporels (TIMESTAMPTZ avec fuseau), des fonctions de fenetrage pour les tendances historiques, et des indexes avances (partiels, GIN). La precision numerique (NUMERIC(10,4)) est mieux respectee que MySQL. De plus, Docker l'image Alpine est plus leger que MariaDB.

Q: Pourquoi ne pas utiliser MongoDB ou une autre base NoSQL ?

R: Les données sont parfaitement relationnelles : les resultats Cucumber ont un schema fixe, les événements de healing ont des relations avec les runs, et les utilisateurs sont typiques d'un systeme d'authentification. Les requêtes d'agrégation (GROUP BY, AVG, SUM) sont natives en SQL. MongoDB aurait ajouté de la complexite sans benefice.

Q: Pourquoi deux modes d'accès (async databases et sync engine) ?

R: FastAPI est asynchrone, donc les repositories utilisant databases.Database ne bloquent pas la boucle d'événements pendant les I/O. Cependant, TestRunRepository utilise un engine synchrone car il est appele depuis des threads standards (threading.Thread) qui ne peuvent pas utiliser les ressources asynchrones de l'API. Le choix est pragmatique et chaque mode est utilisé dans le contexte le plus adapte.

Q: Pourquoi utiliser SQLAlchemy Core plutot que l'ORM complet dans les repositories ?

R: Les repositories utilisent SQLAlchemy Core (sqlalchemy.select(), table.insert(), etc.) plutot que l'ORM (session, unit of work) car les operations sont simples et directes. L'ORM aurait ajouté un surcote de performance et une complexite inutile pour des insertions et des select basiques. Les modèles (DeclarativeBase) sont uniquement utilisés pour la definition des tables et la création du schema.

7.2 Questions sur le schema et la modelisation

Q: Pourquoi les colonnes run_id sont partagees entre 3 tables sans cle etrangere explicite ?

R: C'est un choix delibere de simplicité : les run_id sont des chaînes de caracteres générées par le TestRunner Python (UUID hex 12 caracteres) ou par le CI/CD. L'absence de contrainte FOREIGN KEY permet une flexibilité : les données peuvent arriver de sources différentes (tests locaux, CI/CD, Colab) sans ordre strict. Les indexes BTREE sur run_id compensent le manque de contrainte pour les performances.

Q: Comment est garanti le type des colonnes NUMERIC si SQLite ne les supporte pas ?

R: SQLite stocke tous les types numériques dans un format dynamique (pas de distinction INTEGER/FLOAT/NUMERIC). Le modèle Python declare NUMERIC(10,4) mais SQLAlchemy ne force pas la precision sur SQLite. C'est acceptable car SQLite est uniquement utilisé pour les tests unitaires, pas en production. En production PostgreSQL, le type NUMERIC(10,4) est strictement respecte avec arrondi a 4 decimales.

Q: Pourquoi la colonne logs de test_runs est un TEXT contenant du JSON et pas une table séparée ?

R: Cette decision simplifie le modèle et evite une jointure supplementaire a chaque lecture. Les logs sont rarement consultes individuellement (toujours en bloc avec le run). Cependant, cela rend les requêtes d'indexation ou de recherche dans les logs impossibles au niveau SQL. Pour une version future, une table separate logs serait souhaitable si la recherche dans les logs devient un besoin.

Q: Quelle est la difference entre classification et status dans cucumber_runs ?

R: status est le resultat brut du premier run (passed/failed/skipped). classification est le resultat après re-exécution (rerun) determine par TestClassifier : 'passed' (passe du premier coup), 'flaky' (echoue puis passe au rerun), 'failed' (echoue aux deux runs). La

separation permet de suivre l'historique des tests flaky.

7.3 Questions sur les performances

Q: Quel est le volume de données attendu et comment evolue-t-il ?

R: healing_events : 1 a 10 événements par test, 13 suites x ~50 scénarios = 650 runs par CI/CD, soit 650 a 6500 événements par pipeline. metrics_snapshots : 1 snapshot par run Colab. cucumber_runs : ~50 scénarios/suite x 13 suites = 650 lignes par run. A raison de 10 pipelines/jour, on obtient ~6500 lignes/jour pour healing_events. Aucun problème de performance pour PostgreSQL 16 avec les indexes en place.

Q: Les indexes (DESC sur created_at) sont-ils vraiment utilisés ?

R: Oui. PostgreSQL utilisé un Index Scan Backward sur un index BTREE standard (ASC) pour les ORDER BY DESC, ce qui est aussi efficace qu'un index DESC natif. Les indexes DESC sont declares explicitement dans le schema.sql pour une question de clarte, mais le comportement est identique. Les requêtes de type 'get_latest' sont executees en $O(\log n)$ au lieu de $O(n \log n)$ pour un seq scan + sort.

Q: Comment est calculee la limite de 50 points pour l'historique des métriques ?

R: C'est un choix arbitraire pour le dashboard. 50 points suffisent pour visualiser une tendance sans surcharger le navigateur. La requête utilisé LIMIT 50 avec un ORDER BY captured_at ASC, ce qui est efficace avec l'index. Si l'utilisateur souhaite plus d'historique, le parametre limit peut etre augmente sans impact majeur.

Q: Y a-t-il des risques de contention sur la table healing_events ?

R: Non, car les insertions se font depuis le CI/CD (un seul thread a la fois, suite séquentielle) ou depuis un utilisateur local. En mode multi-utilisateur, les insertions sont rapides et PostgreSQL gere bien la concurrence en ecriture. Les lectures sont des SELECT agregeant qui sont bloquantes mais très rapides avec les indexes.

7.4 Questions sur les migrations et l'evolution

Q: Pourquoi ne pas utiliser Alembic pour les migrations ?

R: Alembic est excellent mais introduit une complexite (fichiers de migration, gestion des versions, commandes upgrade/downgrade) qui n'est pas justifiee par la taille du projet. Le nombre de tables (5) et la frequence des changements de schema (rare) rendent l'approche ALTER TABLE IF NOT EXISTS suffisante. Pour un projet d'entreprise, Alembic serait recommande.

Q: Comment gerer un rollback de migration ?

R: Actuellement, les migrations sont uniquement additives (ADD COLUMN, CREATE INDEX). Un rollback consisterait a supprimer les colonnes ou indexes ajoutes, ce qui n'est pas automatise. C'est un point d'amelioration : ajouter des versions de migration avec Alembic permettrait de monter et descendre entre versions.

Q: Comment la base est-elle versionnée avec le code ?

R: Le schema.sql est stocke dans le repertoire db/ du projet et versionné dans Git. La methode ensure_schema_compatibility est dans le code Python. Les deux sont deployes ensemble via Docker. Il n'y a pas de versionnage explicite du schema, ce qui est une limite : on ne peut pas savoir quelle version du schema correspond a quel commit. Un fichier de version (ex: schema_version.txt) serait utile.

Q: *Comment evoluerait les besoins de stockage avec l'historique des tendances ?

R: Si on souhaite conserver un historique long (6 mois, 1 an), il faudrait soit augmenter la limite de 50 points (ce qui impacterait les perfs du dashboard), soit implementer une retention policy (nettoyage des données > 3 mois), soit ajouter une table de métriques agrégées par jour/semaine avec des vues materialisees pour les tendances. Actuellement, l'historique est limite aux 50 derniers snapshots, ce qui represente environ 50 pipelines.

7.5 Questions sur les tests et la fiabilite

Q: Comment les tests unitaires utilisent-ils la base de données ?

R: Les tests utilisent un fichier SQLite dedie (test_auth.db pour l'authentification). L'URL de connexion contient 'sqlite' ce qui active le

fallback SQLite. Les tables sont creees automatiquement par SQLAlchemy, sans migration. Chaque test peut créer et supprimer des données sans impact sur la base de production.

Q: Comment est assuree la fiabilite des données en cas de crash ?

R: Trois mécanismes : 1) PostgreSQL est configuré avec WAL (Write-Ahead Logging) pour la récupération après crash. 2) TestRunRepository utilisé des transactions explicites (with engine.begin() as conn) qui rollback automatiquement en cas d'erreur. 3) Les données sont dupliques : mémoire (_test_runs) + BDD (test_runs table), ce qui permet de resister a un crash du processus Python sans perdre les runs complètes.

Q: Comment tester le comportement asynchrone des repositories ?

R: Les tests utilisent pytest-asyncio avec asyncio_mode = 'auto' dans pyproject.toml. Les fixtures creent un engine SQLite en mémoire, initialisent les tables, et les repositories sont testes avec des coroutines. Exemple : await repository.create_scenario(...) puis await repository.get_recent(...) -> assert.

Q: *Y a-t-il des tests d'integration qui verifient les migrations ?

R: Actuellement non. Les migrations dynamiques (ensure_schema_compatibility) sont testees manuellement. Un test d'integration ideal creerait une base vierge, appliquerait les migrations, verifierait la presence des colonnes et indexes, puis executerait des insertions/lectures pour valider le schema final.

7.6 Questions avancees (niveau expert)

Q: *Pourquoi le TestRunRepository utilisé-t-il un upsert manuel (SELECT + INSERT/UPDATE) plutot qu'un vrai UPSERT PostgreSQL ?

R: Le choix de l'upsert manuel permet de rester compatible avec SQLite pour les tests. SQLite supporte INSERT OR REPLACE mais avec un comportement différent (suppression puis insertion, ce qui reset les colonnes non fournies). PostgreSQL supporte ON CONFLICT (run_id) DO UPDATE SET ... qui serait plus efficace. Une amelioration serait d'utiliser le dialecte SQLAlchemy avec un insert().on_conflict_do_update() qui s'adapte automatiquement au dialecte.

Q: *Comment sont geres les TIMESTAMPTZ entre Python (avec timezone) et PostgreSQL ?

R: Les modèles utilisent DateTime(timezone=True) qui stocke les timestamps avec timezone (TIMESTAMPTZ en PostgreSQL). Python fournit datetime.now(timezone.utc) pour les nouvelles dates. A la lecture, databases.Database retourné des datetime avec timezone, et les services les serialisent en ISO 8601 via .isoformat(). C'est coherent, mais il faut etre vigilant : Python naive datetime vs aware datetime peut causer des bugs subtils si tous les appels n'utilisent pas timezone.utc.

Q: *Quel est l'impact de l'utilisation de databases.Database (asyncio) sur les transactions ?

R: databases.Database supporte les transactions via async with database.transaction(): mais le projet ne les utilisé pas systematiquement. Chaque insert/select est une transaction implicite (autocommit). Cela peut poser problème en cas d'insertion en masse : si une insertion echoue, les précédentes ne sont pas rollbackees. Pour un usage critique, les insertions de scénarios Cucumber devraient etre dans une transaction explicite pour garantir l'atomicite du run complet.

Q: *Quelle strategie de cache pourrait etre ajoutée pour les requêtes fréquentes ?

R: Deux niveaux de cache pourraient etre ajoutes : 1) Redis en mémoire pour les requêtes les plus fréquentes (get_latest_metrics, get_counts_by_run_id) avec une expiration de quelques secondes. 2) Vues materialisees PostgreSQL pour les aggregations lourdes (healing_rate moyen par jour, tendances hebdomadaires). Actuellement, le seul cache est le healing-baseline.json côté Java pour les locators, pas pour les données du dashboard.

Q: *Si vous deviez partitionner healing_events, comment le feriez-vous ?

R: Le choix le plus naturel serait un partitionnement par plage de dates (range partitioning) sur created_at, par exemple un partition par mois. Cela permettrait de : 1) supprimer rapidement les vieilles partitions (DROP TABLE au lieu de DELETE lent). 2) ameliorer les performances des requêtes recentes (pruning de partition). 3) stocker les partitions recentes sur du stockage rapide et les anciennes sur du stockage lent. L'index run_id serait local a chaque partition.